DATE: 26/09/2026

Name: Hasnain Qazi
SapID: 82739
Program: BS(AI)
Subject: Data Structures and Algorithm

Assignment #1

Q1: Define Data Structure and Abstract Data Type (ADT). Explain their importance in problem-solving with one simple example from a hospital management system.

Ans: Data Structure:

A Data Structure is a way of storing and organizing data in a computer so that it can be accessed and managed easily.

Example's are: Array, LinkedList, Stack, Queue, Tree, Graph.

Abstract Data Type (ADT):

An Abstract Data Type (ADT) defines the type of data and the operations that can be performed on it without showing how those operations are implemented internally.

For example, a list ADT provides operations like insert, remove, search, retrieve and traverse.

Importance:

Data Structures and ADTs help us:

→ organize data properly.
→ search and update data easily.
→ make programs easier to understand.
→ solve problems efficiently

Hospital Example:

DATE: __/__/____

In a hospital management system, patient records can be stored in a list ADT.

For example:

Patient ID: 101

Name: Ali ~~ID: 101~~

Age: 30

Disease: Fever

Room: 12

The system can add, search, remove, or display patient records.

Q2: Define List ADT. Explain its common operations (insert, remove, retrieve, search) and how they help manage patient records.

Ans: A List ADT is an ordered collection of elements where data can be added, removed, searched, retrieved and visited. In a hospital system, the list can be used to store patient records.

1- Insert:

Insert is used to add a new element to the list.

Example: Add a newly admitted patient.

PatientID 101

2- Remove:

Remove deletes an element from the list.

Example: Remove a patient when the patient is discharged.

3- Retrieve:

Retrieve gets the information of a particular patient.

Example: Retrieve the details of Patient ID 101

4- Traverse:

Traverse means visiting every element of the list one by one.

DATE : __/__/____

Example: Display all patients currently admitted in the hospital.

5. Search:

Search is used to find a particular patient.

Example: Search for a patient using their ID.

These list ADT operations make patient record management easier and more organized.

Q3 Design a Hospital Management System using OOP Concepts

Ans:

For this system, we can create a base class called Person. Then we can create a two derived classes: Patient, Doctor.

Base Class: Person

```
{ private:
    string name;
    int id;
    int age;
  public:
    Person(string n, int i, int a)
    { name = n;
      id = i;
      age = a;
    }
    void displayPerson()
    { cout<<"Name:"<<name<<endl;
      cout<<"ID:"<<id<<endl;
      cout<<"Age:"<<age<<endl;
    } };
```

Patient class

```
class patient : public Person
```

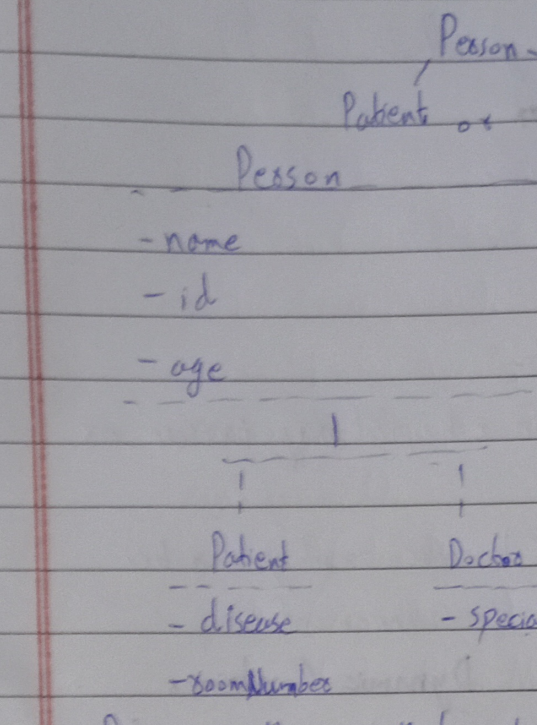
```
private:
    string disease;
    int roomNumber;
public:
    Patient(string n, int i, int a, string d, int r)
        : Person(n,i,a)
    {   disease = d;
        roomNumber = r;
    }
    void displayPatient()
    {   displayPerson();
        cout<< "Disease: " << disease << endl;
        cout<< "Room Number: " << roomNumber << endl;
    }};
```

Doctor class

```
class Doctor : public Person
{ private:
    string specification;
  public:
    Doctor(string n, int i, int a, string s)
        : Person(n,i,a)
    {   specification = s; }
    void displayDoctor() {
        displayPerson();
        cout<< "Specialization: " << specialization << endl;
    }};
```

Class Hierarchy Diagram:

DATE: ___/___/_____



→ Person is the parent or base class.
→ Patient and doctor are child or derived classes.
→ Common information used from Person are: name, id, age

Q4. Explain Dynamic Container Class using pointers and dynamic memory.

Ans:

A Dynamic container class stores data using pointers and dynamic memory.
The number of hospital patients is not always fixed, so dynamic memory is usefull.
A pointer can be created like this:

```
Patient* patient;
```

Memory can then be allocated using new:

```
patients = new Patient[size];
```

new creates memory during program execution.
when the memory is no longer needed, it should be released using:

```
delete[] patients;
```

DATE: __/__/____

This prevents memory from being wasted.

Why is it better than a static array?

A static array has a fixed size.

Example:

```
Patient patient[100];
```

It can only store 100 patients.

But the number of patients in a hospital may increase or decrease.

A dynamic container is better because memory can be allocated according to the program's requirement.

Static Array	Dynamic Container
Fixed size	Size decided at runtime
May waste memory	Better memory usage
Cannot easily changed size	Can be resized
Simply to use	Uses pointers
Memory fixed beforehand	Uses new and delete

Q5: Write pseudocode for List ADT operations.

Ans: **1- Add a New Patient:**

```
void addPatient()
{  if(count == capacity)
   {  resize();
   string name;
   string disease;
   int id;
   int age;
   int room;
```

DATE: __/__/__

```
cout<< "\nEnter Patient ID:";
cin>> id;
cin.ignore();
cout<<"\n Enter Patient Name:";
getline (cin, name);
cout<<" Enter Age:";
cin>>age;
cin.ignore();
cout<<"Enter Disease:";
getline (cin, disease);
cout<<"Enter Room Number:";
cin>> room;
Patient newPatient(
        name,
        id,
        age,
        disease,
        room,
      );
Patients[count] = newPatient;
count++;
cout<<"\n Patient added successfully:"<<endl;
}
```

2- Search Patient by ID:

```
void searchPatient()
{ int searchId;
  cout<<"\nEnter Patient ID to search:";
  cin>> searchID;
```

DATE: ___/___/_____

```
for(i=0; i<count; i++)
{ if(patients[i].getID() == searchID)
  { cout << "\n Patient found:" << endl;
    patients[i].displayPatient();
    return;
  }}
cout << "\n Patient not found." << endl;
}
```

3- ~~Search~~ Remove Patient by ID:

```
void removePatient()
{ int removeId;
  cout << "\n Enter patient ID to remove:";
  cin >> removeID;
  for (int i=0; i<count; i++)
  { if (patients[i].getID() == removeId)
    { for(int j=i; j<count-1; j++)
      { patients[j] = patients[j+1];
      }
      count--;
    cout << "\n Patient removed successfully:" << endl;
    return;
    }
  }
  cout << "\n Patient not found" << endl;
}

//
```

D:\University\Riphah University\3rd Semester\Data Structures\Assignments\Assignment no1\Untitled1.exe

```
===== Hospital Management System =====
1. Add Patient
2. Remove Patient
3. Search Patient
4. Display All Patients
5. Exit
Enter choice: 1

Enter Patient ID: 101
Enter Patient Name: Ahmad
Enter Age: 18
Enter Disease: Fever
Enter Room Number: 216

Patient added successfully.

===== Hospital Management System =====
1. Add Patient
2. Remove Patient
3. Search Patient
4. Display All Patients
5. Exit
Enter choice: 4

===== Display All Patients =====
Name: Ahmad | ID: 101 | Age: 18 | Disease: Fever | Room: 216

===== Hospital Management System =====
1. Add Patient
2. Remove Patient
3. Search Patient
4. Display All Patients
5. Exit
Enter choice: 3

Enter Patient ID to search: 101

Patient Found:
Name: Ahmad | ID: 101 | Age: 18 | Disease: Fever | Room: 216

===== Hospital Management System =====
1. Add Patient
2. Remove Patient
3. Search Patient
4. Display All Patients
5. Exit
Enter choice: 2

Enter Patient ID to remove: 101

Patient removed successfully.
```